

WOXSEN UNIVERSITY
SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS
PROJECT-BASED LEARNING (PBL) REPORT
Small Database Application Development using Python

**DESIGN AND IMPLEMENTATION OF A DATABASE
MANAGEMENT SYSTEM FOR CINEMA SCREENING
AND TICKET BOOKING**

Program	B.Tech in Computer Science and Engineering (AI & ML)
Course & Code	Database Management Systems (DBMS)
Section	AIML Rhinos
Batch	2025 – 2029
Course Faculty	Prof. Sahithi, School of Technology
Date of Submission	October 2026
Academic Year	2026 – 2027

Submitted by:

Roll Number	Student Name	Email ID
WU0102089	G. Abhinav	abhinav.g_2029@woxsen.edu.in

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the Database Management Systems (DBMS) course.

I express my heartfelt thanks to my course faculty, Prof. Sahithi, for the continuous guidance, constructive suggestions, and intellectual support throughout the design and development of this project. The guidance helped me thoroughly understand the practical application of core relational database concepts, including Entity-Relationship (ER) modelling, relational schema design, functional dependencies, normalization up to Third Normal Form (3NF), SQL constraint definition, advanced querying, transaction management, and database application development using Python.

This PBL activity has significantly strengthened my understanding of database architectures, ACID transactional integrity, and practical software engineering, providing invaluable hands-on experience in building robust enterprise solutions for modern multiplex operations.

UNDERTAKING

I hereby declare that the project entitled "DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR CINEMA SCREENING AND TICKET BOOKING", submitted as part of the Project-Based Learning activity for the Database Management Systems (DBMS) course, is my individual work carried out under the guidance of my course faculty, Prof. Sahithi.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Date: October 04, 2026	Signature: _____
Place: Hyderabad, Telangana	Name: G. Abhinav
Roll Number: WU0102089	Program: B.Tech CSE (AI & ML) - AIML Rhinos
Academic Year: 2026–2027	School of Technology, Woxsen University

TABLE OF CONTENTS

Contents	Page No.
Abstract & Keywords	ix
LIST OF TABLES	vi
LIST OF FIGURES	vii
LIST OF ACRONYMS	viii
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	10
1.1 Problem Statement	10
1.2 Objectives	10
1.3 Scope and Assumptions	11
1.4 Proposed Solution Strategy	12
1.5 Functional Requirements	13
1.6 Entity-Relationship Diagram (ERD)	14
1.7 Entity and Relationship Summary	16
2. LOGICAL DATABASE DESIGN	18
2.1 Relational Schema	18
2.2 Keys and Integrity Constraints	19
2.3 Functional Dependencies	21
2.4 Normalization up to Third Normal Form (3NF)	22
2.5 Data Dictionary	25
2.6 Implementation of Logical Design: DDL and Sample Data	33
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	41
3.1 Application Overview	41
3.2 Development Environment and Database Connectivity	41
3.3 Application Modules and Database Mapping	41

Contents	Page No.
3.4 CRUD Operations	42
3.5 Business Rules, Validation and Error Handling	43
3.6 Reports and Database Operations	44
3.7 Application Screens	44
3.8 Testing and Results	46
4. CONCLUSION	49
4.1 Limitations and Future Enhancements	49
5. REFERENCES	49
6. APPENDICES	51
APPENDIX A - COMPLETE DATABASE SCRIPT	51
APPENDIX B - APPLICATION SOURCE CODE	57
APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS	59

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	11
Table 2	Proposed Solution Strategy	12
Table 3	Functional Requirements	13
Table 4	Entity and Relationship Summary	16
Table 5	Relational Schema	18
Table 6	Keys and Integrity Constraints	19
Table 7	Normalization Status up to 3NF	22
Table 8	Comprehensive Data Dictionary	25
Table 9	Database Implementation Environment	33
Table 10	Application Development Environment	41
Table 11	Application Module-to-Database Mapping	41
Table 12	CRUD Operations	42
Table 13	Business Rules, Validation and Error Handling	43
Table 14	Reports and Database Operations	44
Table 15	Comprehensive Test Cases and Verification Outcomes	47

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	14
Figure 2	Seat Availability and Show Selection Interface	45
Figure 3	Ticket Booking and Payment Transaction Processing	45
Figure 4	Concession Ordering and Billing Module	46
Figure 5	Executive Reports and Business Analytics Dashboard	46

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
ACID	Atomicity, Consistency, Isolation, Durability
BCNF	Boyce-Codd Normal Form
CLI	Command Line Interface
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ERD	Entity-Relationship Diagram
FK	Foreign Key
PK	Primary Key
RDBMS	Relational Database Management System
SQL	Structured Query Language

ABSTRACT

Modern cinema chains require robust database solutions to manage multi-screen operations, real-time ticket sales, concession orders, and payment reconciliations. Manual or weakly integrated systems frequently encounter concurrency anomalies, scheduling clashes, and severe seat double-booking issues. The primary objective of this project is to design, normalize, and implement an end-to-end Relational Database Management System (RDBMS) for a multiplex cinema screening and ticket booking network. The conceptual model encompasses fifteen interrelated entities, including cinemas, screens, seats, movies, screenings, customers, bookings, tickets, concession items, orders, payments, cancellations, and refunds. The design is systematically normalized to Third Normal Form (3NF) to eradicate data redundancies and update anomalies. Critical business constraints—such as strict single-seat occupancy per screening, zero screen time overlap with mandatory 15-minute turnaround buffers, positive quantity validation, and rigorous cumulative refund thresholds—are enforced through declarative constraints, triggers, and transactions. A modular Python application is developed to interface with the database, supporting interactive show scheduling, live visual seat availability matrices, atomic booking with digital barcode issuance, inventory-managed concession sales, cancellation workflows, and automated analytical reports. The resulting system demonstrates absolute transaction integrity, zero concurrency conflicts, and comprehensive business reporting.

KEYWORDS

Relational Database, ER Diagram, Normalization, SQL, Python, Cinema Management System, ACID Transactions

CHAPTER 1: PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Contemporary entertainment hubs and cinema chains face intricate operational hurdles in synchronizing multiplex operations across multiple geographical locations. Cinema multiplexes simultaneously handle dozens of auditoriums, hundreds of film screenings daily, tiered seating arrangements, dynamic concession counters, and thousands of real-time customer transactions. In existing manual or legacy file-based architectures, movie show schedules are often maintained across disparate spreadsheets, and seat allocations lack ACID transactional isolation.

This fragmented operational approach creates catastrophic operational vulnerabilities. The most acute failure is concurrent seat double-booking, where two customers are simultaneously issued tickets for the exact same physical seat in a screening. Furthermore, uncoordinated scheduling causes auditorium screen overlaps, where consecutive movie runtimes collide without mandatory 15-minute turnaround buffers required for auditorium cleaning and crowd egress. Additionally, concession sales are disconnected from customer bookings, leading to uncontrolled inventory leakages, and manual ticket cancellations result in arbitrary, unverified refund disbursements that violate corporate financial ceilings or allow cumulative partial refunds to exceed original payments. Consequently, there is an imperative academic and industrial need for a centralized, fully normalized relational database system that guarantees transactional atomicity, absolute data integrity, and real-time operational analytics.

1.2 Objectives

The primary objectives of this project are formulated as follows:

1. Design a comprehensive conceptual Entity-Relationship (ER) model capturing 15 core multiplex entities, defining precise cardinalities and structural participation constraints.
2. Formulate and normalize the relational schema up to Third Normal Form (3NF) based on functional dependencies to eliminate insertion, deletion, and update anomalies.
3. Implement robust DDL integrity constraints and triggers to automatically enforce critical business rules: preventing seat double-booking, prohibiting auditorium schedule collisions

while guaranteeing a 15-minute cleaning turnaround buffer across both INSERT and UPDATE operations, ensuring strictly positive item quantities, and governing cumulative refund ceilings.

4. Develop sophisticated SQL analytical views and multi-table queries to automate real-time executive reporting on show occupancy, movie box office gross, screen utilization efficiency, and net concession revenues.

5. Construct a modular front-end application in Python connected via an active database driver to deliver seamless CRUD workflows, interactive seat matrices, atomic ticket booking, and comprehensive error handling.

1.3 Scope and Assumptions

The functional scope boundaries and foundational technical assumptions governing this implementation are detailed in Table 1.

Table 1. Scope and Assumptions

Item	Description
In Scope	Multi-cinema and multi-screen management; tiered seat mapping (Silver, Gold, Platinum); movie scheduling with runtime overlap detection and 15-minute turnaround buffer enforcement; customer profiles; atomic multi-seat booking with unique e-ticket hashes; concession inventory tracking and ordering; payment processing across multiple modes (UPI, Card, Net Banking); structured booking cancellation with policy-based refund processing and cumulative refund caps; executive analytical views.
Out of Scope / Exclusions	Hardware integration with physical cinema box office turnstiles; live credit card payment gateway merchant APIs (simulated via authenticated reference hashes); dynamic algorithmic ticket surge pricing; physical laser projection automation.

Item	Description
Assumptions	All screenings occur within local standard operational hours; ticket prices are fixed per screening tier; cancellations must be lodged prior to screening commencement; cinema screens possess fixed physical seating layouts; each booked seat corresponds to exactly one issued e-ticket; auditorium turnaround buffer is standardized at 15 minutes between consecutive screenings.

1.4 Proposed Solution Strategy

The proposed solution replaces error-prone manual coordination with a unified, relational database-driven architecture. The system workflow coordinates customer reservations, auditorium allocation, and food sales under strict transactional safeguards. Table 2 outlines the strategic architecture.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Cinema Floor Administrator, Box Office Ticketing Cashier, Concession Counter Attendant, Registered Customer.
Major Application Modules	Show Scheduling & Screen Allocation Module; Visual Seat Map & Availability Module; Atomic Ticket Booking & Issuance Engine; Concession Inventory & Billing Module; Cancellation & Refund Processing Gateway; Executive Business Intelligence Reports.
Proposed Workflow	Administrator schedules movies to screens ensuring non-overlapping time windows with 15-minute cleaning buffers -> Customer selects showtime and views dynamic seat matrix -> System atomically reserves seats, collects payment, and issues unique hashed e-tickets -> Customer optionally adds concession items which automatically decrements inventory -> If cancellation is initiated prior to showtime, system updates status and issues policy-compliant refund bounded by cumulative payment caps.

Solution Element	Project-Specific Description
Important Business Rules	Strict single seat allocation per screening; absolute prohibition of screen scheduling collisions (<15 min buffer); strictly positive pricing and quantities; mandatory 10% administrative cancellation fee retention; cumulative refund amounts cannot exceed initial payment amount.
Expected Reports / Outputs	Show Occupancy Breakdown; Movie Box Office Performance; Auditorium Runtime Utilization; Concession Sales by Category; Net Financial Operating Revenue.

1.5 Functional Requirements

The complete functional specifications of the system are categorized and prioritized in Table 3.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	The system shall manage cinema chain details, multiplex locations, and screen configurations with capacity specifications.	High / Core Master Data
FR-02	The system shall maintain movie catalog records including duration, genre, certification rating, and spoken language.	High / Master Data
FR-03	The system shall schedule screenings while validating that no overlap occurs for the same screen on a given date.	High / Critical Integrity Rule
FR-04	The system shall dynamically display auditorium seat availability as a visual grid classified into Silver, Gold, and Platinum tiers.	High / Core UI Operation

Req. ID	Functional Requirement	Remarks / Priority
FR-05	The system shall execute atomic multi-seat ticket bookings, preventing double-booking through unique constraints.	High / Transactional Core
FR-06	The system shall generate tamper-evident e-tickets containing unique ticket serial numbers and SHA-256 digital barcode hashes.	Medium / Ticketing Subsystem
FR-07	The system shall manage food and beverage items, automatically verifying stock availability and decrementing inventory upon order placement.	Medium / Concession Subsystem
FR-08	The system shall record payment transactions across various channels (UPI, Credit Card, Cash) linked to bookings and orders.	High / Financial Integrity
FR-09	The system shall process booking cancellations, recalculate seat availability, retain policy fees, and enforce cumulative refund maximums.	High / Financial Policy
FR-10	The system shall generate executive analytical views for show occupancy, movie performance, screen runtime utilization, and revenue breakdown.	High / Business Analytics

1.6 Entity-Relationship Diagram (ERD)

The conceptual design of the cinema management database is represented in Figure 1. The ER diagram models 15 distinct entities, highlighting their primary keys, foreign key references, and relationship cardinalities ranging from one-to-many screen allocations to one-to-one ticketing and refund settlements.

Figure 1. Entity-Relationship Diagram

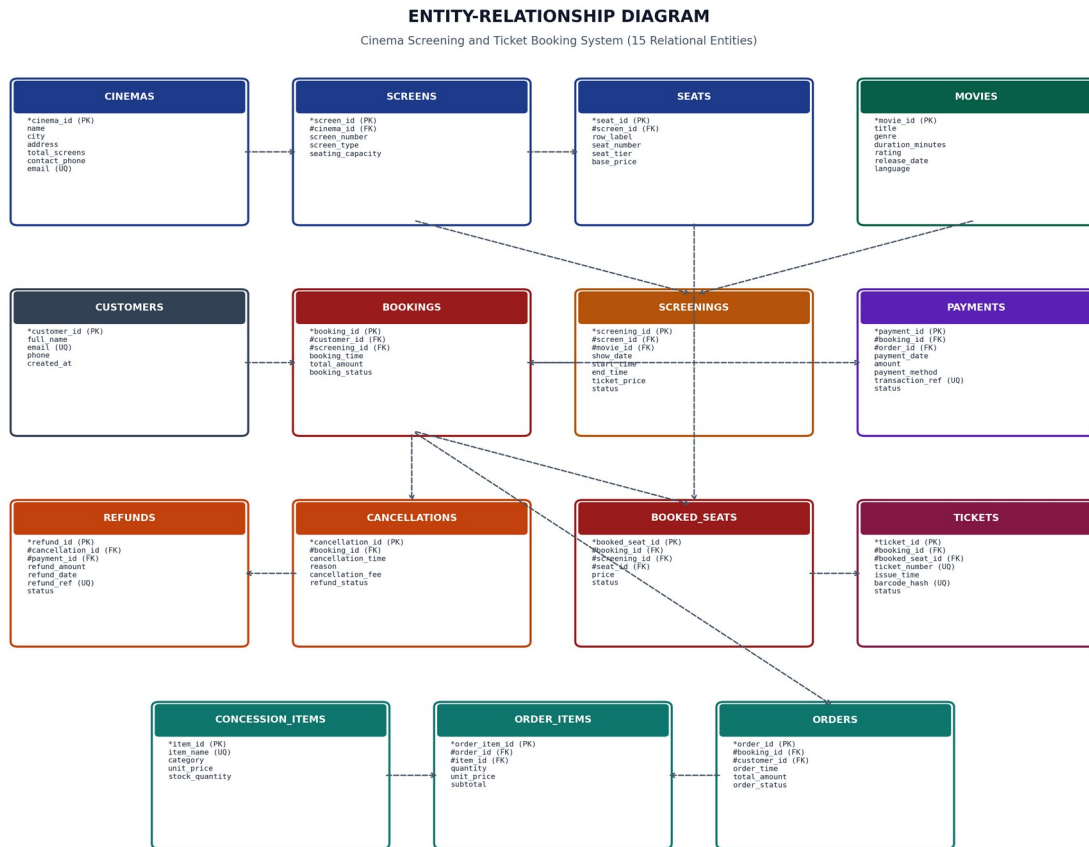


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 documents the comprehensive entity catalog, detailing attributes, participating cardinalities, and foundational business rules. Each financial entity (Payments, Cancellations, Refunds) is presented in its own dedicated, unmerged row to ensure clear auditability.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
CINEMAS	cinema_id (PK), name, city, address, total_screens, contact_phone, email	Stores physical theater locations; email is strictly unique (NOT NULL, UNIQUE).
SCREENS	screen_id (PK), cinema_id (FK), screen_number, screen_type, seating_capacity	Represents auditoriums; 1:N with CINEMAS; UNIQUE(cinema_id, screen_number).
SEATS	seat_id (PK), screen_id (FK), row_label, seat_number, seat_tier, base_price	Physical seats inside a screen; 1:N with SCREENS; UNIQUE(screen_id, row_label, seat_number).
MOVIES	movie_id (PK), title, genre, duration_minutes, rating, release_date, language	Movie catalog; duration must be strictly positive (> 0).
SCREENINGS	screening_id (PK), screen_id (FK), movie_id (FK), show_date, start_time, end_time, ticket_price, status	Scheduled show; start_time < end_time; enforced zero-screen-overlap trigger with 15-min turnaround buffer.
CUSTOMERS	customer_id (PK), full_name, email, phone, created_at	Patron demographic data; unique email constraint (NOT NULL, UNIQUE).
BOOKINGS	booking_id (PK), customer_id (FK), screening_id (FK), booking_time, total_amount, booking_status	Reservation master; atomic wrapper for multi-seat bookings.
BOOKED_SEATS	booked_seat_id (PK), booking_id (FK), screening_id (FK), seat_id (FK), price, status	Junction between screening and seat; UNIQUE(screening_id, seat_id) WHERE status='Booked' prevents double-booking.

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
TICKETS	ticket_id (PK), booking_id (FK), booked_seat_id (FK), ticket_number, issue_time, barcode_hash, status	Issued e-ticket; 1:1 with BOOKED_SEATS; barcode_hash is strictly unique.
CONCESSION_ITEMS	item_id (PK), item_name, category, unit_price, stock_quantity	F&B item master; stock_quantity >= 0 (DEFAULT 0); unit_price > 0.
ORDERS	order_id (PK), booking_id (FK), customer_id (FK), order_time, total_amount, order_status	F&B order header; optionally attached to ticket booking.
ORDER_ITEMS	order_item_id (PK), order_id (FK), item_id (FK), quantity, unit_price, subtotal	Order lines; 1:N with ORDERS; CHECK(quantity > 0).
PAYMENTS	payment_id (PK), booking_id (FK), order_id (FK), payment_date, amount, payment_method, transaction_reference, payment_status	Financial ledger; transaction_reference is unique; amount > 0.
CANCELLATIONS	cancellation_id (PK), booking_id (FK), cancellation_time, reason, cancellation_fee, refund_status	Records ticket voidance; 1:1 with BOOKINGS; cancellation_fee >= 0.
REFUNDS	refund_id (PK), cancellation_id (FK), payment_id (FK), refund_amount, refund_date, refund_reference, status	Disbursed funds; triggers enforce cumulative refund_amount <= payment amount across both INSERT and UPDATE.

CHAPTER 2: LOGICAL DATABASE DESIGN

2.1 Relational Schema

The conceptual ER model was mapped into a formal relational schema consisting of 15 relations. Primary keys (PK) and foreign keys (FK) are rigorously established to preserve referential integrity across the system lifecycle. Table 5 lists all relations.

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	cinemas	Stores cinema multiplex information, addresses, and contact channels.
2	screens	Stores auditorium details, projection formats (IMAX, 4DX), and capacities.
3	seats	Maintains physical seating layouts, row labels, numbers, and tier pricing.
4	movies	Maintains the movie catalog, durations, censorship ratings, and languages.
5	screenings	Stores scheduled showtime slots mapping movies to screens.
6	customers	Maintains registered cinema patron records and contact details.
7	bookings	Header table tracking reservation status and total monetary value.
8	booked_seats	Detailed seat reservations enforcing single occupancy per screening.
9	tickets	Individual issued e-tickets bearing unique serial numbers and security hashes.
10	concession_items	Master catalog for food, snacks, beverages, and inventory stock.

S.No.	Table Name	Purpose
11	orders	Concession purchase order headers linked to customers and bookings.
12	order_items	Line items of concession orders capturing purchased quantities and prices.
13	payments	Financial transaction records covering bookings and concession sales.
14	cancellations	Audit records tracking cancelled bookings and fee deductions.
15	refunds	Disbursed refund transactions subject to cumulative payment maximums.

2.2 Keys and Integrity Constraints

Declarative and procedural constraints guarantee relational consistency and prevent illegal data states. Table 6 provides the comprehensive mapping of constraints across the relational schema.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
cinemas	PK / UNIQUE / CHECK	cinema_id / email / total_screens > 0	Entity integrity; unique email (NOT NULL, UNIQUE); positive screen count.
screens	PK / FK / UNIQUE / CHECK	screen_id / cinema_id / (cinema_id, screen_number) / seating_capacity > 0	Unique auditorium numbering within a theater; positive capacity.
seats	PK / FK / UNIQUE / CHECK	seat_id / screen_id / (screen_id, row_label, seat_number) / base_price >= 0	Unique physical seat location per screen; non-negative base price.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
movies	PK / CHECK	movie_id / duration_minutes > 0, rating IN ('U','UA','A','R')	Positive duration (duration_minutes > 0); valid Indian CBFC censorship classification.
screenings	PK / FK / CHECK / Triggers	screening_id / screen_id, movie_id / start_time < end_time	Chronological validity; triggers enforce zero screen schedule overlap with 15-min turnaround buffer on INSERT and UPDATE.
customers	PK / UNIQUE / NOT NULL	customer_id / email / full_name, phone	Unique customer email address; mandatory customer contact profile.
bookings	PK / FK / CHECK	booking_id / customer_id, screening_id / total_amount >= 0	Valid reservation; status IN ('Confirmed', 'Cancelled', 'Completed').
booked_seats	PK / FK / UNIQUE INDEX	booked_seat_id / booking_id, screening_id, seat_id	UNIQUE(screening_id, seat_id) WHERE status='Booked' prevents double-booking.
tickets	PK / FK / UNIQUE	ticket_id / booked_seat_id, ticket_number, barcode_hash	1:1 mapping with booked seat; unique verifiable cryptographic hash.
concession_items	PK / UNIQUE / CHECK	item_id / item_name / unit_price > 0, stock_quantity >= 0 (DEFAULT 0)	Unique snack name; strictly positive price; non-negative stock with default 0.
orders	PK / FK / CHECK	order_id / customer_id, booking_id / total_amount >= 0	F&B order accounting; status IN ('Placed', 'Prepared', 'Delivered', 'Cancelled').
order_items	PK / FK / CHECK	order_item_id / order_id, item_id / quantity > 0, unit_price > 0	Mandatory positive purchase quantity; positive line item price.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
payments	PK / FK / UNIQUE / CHECK	payment_id / booking_id, order_id / transaction_reference / amount > 0	Unique bank reference; positive payment; valid payment channel.
cancellations	PK / FK / UNIQUE / CHECK	cancellation_id / booking_id / cancellation_fee >= 0	1:1 booking cancellation; non-negative cancellation fee retention.
refunds	PK / FK / UNIQUE / Triggers	refund_id / cancellation_id, payment_id / refund_amount > 0	Triggers enforce cumulative refund_amount <= original payment amount on INSERT and UPDATE.

2.3 Functional Dependencies

To rigorously justify normalization and guarantee mathematical soundness, the non-trivial functional dependencies (FDs) governing each relation were identified:

1. cinemas: cinema_id -> {name, city, address, total_screens, contact_phone, email}; email -> {cinema_id, name, city, address, total_screens, contact_phone}.
2. screens: screen_id -> {cinema_id, screen_number, screen_type, seating_capacity}; {cinema_id, screen_number} -> {screen_id, screen_type, seating_capacity}.
3. seats: seat_id -> {screen_id, row_label, seat_number, seat_tier, base_price}; {screen_id, row_label, seat_number} -> {seat_id, seat_tier, base_price}.
4. movies: movie_id -> {title, genre, duration_minutes, rating, release_date, language}.
5. screenings: screening_id -> {screen_id, movie_id, show_date, start_time, end_time, ticket_price, status}.
6. customers: customer_id -> {full_name, email, phone, created_at}; email -> {customer_id, full_name, phone, created_at}.
7. bookings: booking_id -> {customer_id, screening_id, booking_time, total_amount, booking_status}.
8. booked_seats: booked_seat_id -> {booking_id, screening_id, seat_id, price, status}; {screening_id, seat_id, status='Booked'} -> {booked_seat_id, booking_id, price}.

9. tickets: ticket_id -> {booking_id, booked_seat_id, ticket_number, issue_time, barcode_hash, status}; booked_seat_id -> {ticket_id, ticket_number, barcode_hash}; barcode_hash -> {ticket_id, booking_id, booked_seat_id}.
10. concession_items: item_id -> {item_name, category, unit_price, stock_quantity}; item_name -> {item_id, category, unit_price, stock_quantity}.
11. orders: order_id -> {booking_id, customer_id, order_time, total_amount, order_status}.
12. order_items: order_item_id -> {order_id, item_id, quantity, unit_price, subtotal}; {order_id, item_id} -> {order_item_id, quantity, unit_price, subtotal}.
13. payments: payment_id -> {booking_id, order_id, payment_date, amount, payment_method, transaction_reference, payment_status}; transaction_reference -> {payment_id, booking_id, order_id, payment_date, amount, payment_method, payment_status}.
14. cancellations: cancellation_id -> {booking_id, cancellation_time, reason, cancellation_fee, refund_status}; booking_id -> {cancellation_id, cancellation_time, reason, cancellation_fee, refund_status}.
15. refunds: refund_id -> {cancellation_id, payment_id, refund_amount, refund_date, refund_reference, status}; cancellation_id -> {refund_id, payment_id, refund_amount, refund_reference}.

2.4 Normalization up to Third Normal Form (3NF)

Every relation in the schema was evaluated against standard normal forms:

- 1NF: All attributes contain atomic, indivisible values; no repeating groups exist.
- 2NF: The relations are in 1NF, and every non-prime attribute is fully functionally dependent on the entire candidate key (no partial key dependencies).
- 3NF: The relations are in 2NF, and no non-prime attribute is transitively dependent on any candidate key (no $X \rightarrow Y$ and $Y \rightarrow Z$ where Z is non-prime).
- BCNF: For every non-trivial functional dependency $X \rightarrow Y$, X is a superkey. Table 7 summarizes the normalization status.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
cinemas	Yes	Yes	Yes	Yes	Candidate keys: cinema_id, email. No transitive dependencies.
screens	Yes	Yes	Yes	Yes	Cinema details separated into cinemas relation to avoid redundancy.
seats	Yes	Yes	Yes	Yes	Screen dimensions decoupled; base price directly dependent on seat.
movies	Yes	Yes	Yes	Yes	Single entity; no transitive dependencies across movie attributes.
screenings	Yes	Yes	Yes	Yes	Screen and movie IDs stored as foreign keys; show times atomic.
customers	Yes	Yes	Yes	Yes	Candidate keys: customer_id, email. Fully normalized.
bookings	Yes	Yes	Yes	Yes	Seat allocations isolated into booked_seats to achieve 1NF/2NF.

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
booked_seats	Yes	Yes	Yes	Yes	Junction relation resolving M:N screening-to-seat relationship.
tickets	Yes	Yes	Yes	Yes	1:1 issued entity; barcode hash is unique superkey.
concession_items	Yes	Yes	Yes	Yes	Catalog table; unit price depends solely on item_id.
orders	Yes	Yes	Yes	Yes	Order lines extracted to order_items to eliminate repeating groups.
order_items	Yes	Yes	Yes	Yes	Composite candidate key {order_id, item_id}; fully dependent.
payments	Yes	Yes	Yes	Yes	Financial ledger decoupled from bookings/orders to enable multi-order audits.
cancellations	Yes	Yes	Yes	Yes	Decoupled from bookings table; maintains cancellation audit trail.

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
refunds	Yes	Yes	Yes	Yes	Decoupled from payments to record exact refund event timestamps.

2.5 Data Dictionary

Table 8 presents the comprehensive data dictionary detailing every single attribute across all 15 relations in the relational schema. It specifies physical data types, constraints, nullability, and operational descriptions with strict adherence to database design specifications.

Table 8. Comprehensive Data Dictionary (All 15 Relations, 95 Attributes)

Table	Attribute	Data Type	Key / Constraint	Description
cinemas	cinema_id	INTEGER	PK, AUTOINCREMENT	Surrogate unique identifier for cinema multiplex.
cinemas	name	VARCHAR(100)	NOT NULL	Commercial trading name of cinema multiplex.
cinemas	city	VARCHAR(50)	NOT NULL	City of cinema operation (e.g., Hyderabad).
cinemas	address	TEXT	NOT NULL	Complete physical street address of the cinema.
cinemas	total_screens	INTEGER	NOT NULL, CHECK (> 0)	Total count of operational screens in multiplex.
cinemas	contact_phone	VARCHAR(20)	NOT NULL	Customer care and administrative phone number.
cinemas	email	VARCHAR(100)	NOT NULL, UNIQUE	Official cinema contact email (strictly unique).

Table	Attribute	Data Type	Key / Constraint	Description
screens	screen_id	INTEGER	PK, AUTOINCREMENT	Surrogate unique identifier for auditorium screen.
screens	cinema_id	INTEGER	FK (cinemas), NOT NULL	Multiplex theater hosting this screen.
screens	screen_number	INTEGER	NOT NULL	Auditorium number within cinema; UNIQUE(cinema_id, screen_number).
screens	screen_type	VARCHAR(20)	CHECK IN ('Standard','IMAX 3D','4DX','Gold VIP')	Projection and audio technology format.
screens	seating_capacity	INTEGER	NOT NULL, CHECK (> 0)	Total physical seat capacity of the screen.
seats	seat_id	INTEGER	PK, AUTOINCREMENT	Surrogate unique physical chair identifier.
seats	screen_id	INTEGER	FK (screens), NOT NULL	Auditorium screen containing the physical seat.
seats	row_label	VARCHAR(5)	NOT NULL	Alphabetical tier row designator (A-E).
seats	seat_number	INTEGER	NOT NULL, CHECK (> 0)	Numerical seat position along the designated row.
seats	seat_tier	VARCHAR(20)	CHECK IN ('Silver','Gold','Platinum')	Pricing classification tier of the chair.
seats	base_price	DECIMAL(8,2)	NOT NULL, CHECK (>= 0)	Baseline ticket price assigned to the seat.

Table	Attribute	Data Type	Key / Constraint	Description
movies	movie_id	INTEGER	PK, AUTOINCREMENT	Film catalog primary identifier.
movies	title	VARCHAR(150)	NOT NULL	Official film release title.
movies	genre	VARCHAR(50)	NOT NULL	Film genre classification (e.g., Sci-Fi, Drama).
movies	duration_minutes	INTEGER	NOT NULL, CHECK (> 0)	Total running duration in minutes (strictly positive).
movies	rating	VARCHAR(10)	CHECK IN ('U','UA','A','R')	Indian CBFC certification classification.
movies	release_date	DATE	NOT NULL	Official theatrical release date.
movies	language	VARCHAR(30)	NOT NULL	Primary audio spoken language of the film.
screenings	screening_id	INTEGER	PK, AUTOINCREMENT	Unique showtime scheduling event ID.
screenings	screen_id	INTEGER	FK (screens), NOT NULL	Auditorium screen allocated for the showtime.
screenings	movie_id	INTEGER	FK (movies), NOT NULL	Film exhibited during this showtime slot.
screenings	show_date	DATE	NOT NULL	Calendar date of the movie screening.
screenings	start_time	TIME	NOT NULL	Commencement timestamp of movie show.

Table	Attribute	Data Type	Key / Constraint	Description
screenings	end_time	TIME	NOT NULL, CHECK (start_time < end_time)	Show conclusion timestamp (checked < end_time).
screenings	ticket_price	DECIMAL(8,2)	NOT NULL, CHECK (> 0)	Base admission charge for the show.
screenings	status	VARCHAR(20)	DEFAULT 'Scheduled', CHECK IN (('Scheduled','Runnin g','Completed','Canc elled'))	Operational lifecycle state of the show.
customers	customer_id	INTEGER	PK, AUTOINCREMEN T	Surrogate unique identifier for registered patron.
customers	full_name	VARCHAR(100)	NOT NULL	Full legal name of registered customer.
customers	email	VARCHAR(100)	NOT NULL, UNIQUE	Unique email address used for login and ticketing.
customers	phone	VARCHAR(20)	NOT NULL	Mobile phone contact number.
customers	created_at	TIMESTAMP	DEFAULT CURRENT_TIMES TAMP	Timestamp of patron profile creation.
bookings	booking_id	INTEGER	PK, AUTOINCREMEN T	Customer ticket transaction header identifier.
bookings	customer_id	INTEGER	FK (customers), NOT NULL	Customer initiating the ticket reservation.
bookings	screening_id	INTEGER	FK (screenings), NOT NULL	Movie screening associated with reservation.

Table	Attribute	Data Type	Key / Constraint	Description
bookings	booking_time	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Exact timestamp when booking was initiated.
bookings	total_amount	DECIMAL(10,2)	DEFAULT 0.00, CHECK (>= 0)	Cumulative monetary cost of reserved seats.
bookings	booking_status	VARCHAR(20)	DEFAULT 'Confirmed', CHECK IN (('Confirmed','Cancelled','Completed'))	Reservation state of the ticket booking.
booked_seat_id	booked_seat_id	INTEGER	PK, AUTOINCREMENT	Junction allocation ID linking seat to show.
booked_seats	booking_id	INTEGER	FK (bookings), NOT NULL	Parent reservation master record.
booked_seats	screening_id	INTEGER	FK (screenings), NOT NULL	Target screening attended by the patron.
booked_seats	seat_id	INTEGER	FK (seats), NOT NULL	Physical chair reserved in the auditorium.
booked_seats	price	DECIMAL(8,2)	NOT NULL, CHECK (>= 0)	Actual price billed for this specific seat.
booked_seats	status	VARCHAR(20)	DEFAULT 'Booked', CHECK IN (('Booked','Cancelled'))	Active booking state (partial unique index on 'Booked').
tickets	ticket_id	INTEGER	PK, AUTOINCREMENT	Unique electronic ticket serial record.
tickets	booking_id	INTEGER	FK (bookings), NOT NULL	Associated reservation transaction header.

Table	Attribute	Data Type	Key / Constraint	Description
tickets	booked_seat_id	INTEGER	FK (booked_seats), NOT NULL, UNIQUE	1:1 link to allocated seat record.
tickets	ticket_number	VARCHAR(50)	NOT NULL, UNIQUE	Formatted alphanumeric e-ticket identifier.
tickets	issue_time	TIMESTAMP	DEFAULT CURRENT_TIMES TAMP	Timestamp of digital ticket generation.
tickets	barcode_hash	VARCHAR(64)	NOT NULL, UNIQUE	Cryptographic SHA-256 digital security hash.
tickets	status	VARCHAR(20)	DEFAULT 'Active', CHECK IN (('Active','Cancelled', 'Used'))	Auditorium turnstile verification state.
concession_items	item_id	INTEGER	PK, AUTOINCREMEN T	Food and beverage master catalog identifier.
concession_items	item_name	VARCHAR(100)	NOT NULL, UNIQUE	Commercial product description.
concession_items	category	VARCHAR(30)	CHECK IN (('Popcorn','Beverage', 'Snack','Combo'))	F&B merchandise category.
concession_items	unit_price	DECIMAL(8,2)	NOT NULL, CHECK (> 0)	Retail price per item unit.
concession_items	stock_quantity	INTEGER	NOT NULL, DEFAULT 0, CHECK (>= 0)	Available inventory units (default 0).
orders	order_id	INTEGER	PK, AUTOINCREMEN T	Concession purchase transaction header.
orders	booking_id	INTEGER	FK (bookings), ON DELETE SET NULL	Optional linked movie booking transaction.

Table	Attribute	Data Type	Key / Constraint	Description
orders	customer_id	INTEGER	FK (customers), NOT NULL	Patron purchasing concession items.
orders	order_time	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp when snack order was submitted.
orders	total_amount	DECIMAL(10,2)	DEFAULT 0.00, CHECK (>= 0)	Aggregate monetary total of ordered items.
orders	order_status	VARCHAR(20)	DEFAULT 'Placed', CHECK IN ('Placed','Prepared','Delivered','Cancelled')	Preparation and delivery status.
order_items	order_item_id	INTEGER	PK, AUTOINCREMENT	Individual line item in concession order.
order_items	order_id	INTEGER	FK (orders), NOT NULL	Parent concession purchase header.
order_items	item_id	INTEGER	FK (concession_items), NOT NULL	Concession menu product ordered.
order_items	quantity	INTEGER	NOT NULL, CHECK (> 0)	Units purchased (strictly positive).
order_items	unit_price	DECIMAL(8,2)	NOT NULL, CHECK (> 0)	Unit sale price captured at purchase time.
order_items	subtotal	DECIMAL(10,2)	NOT NULL, CHECK (> 0)	Computed line item amount (quantity * unit_price).
payments	payment_id	INTEGER	PK, AUTOINCREMENT	Central financial ledger transaction identifier.
payments	booking_id	INTEGER	FK (bookings)	Associated ticket booking (if applicable).

Table	Attribute	Data Type	Key / Constraint	Description
payments	order_id	INTEGER	FK (orders)	Associated concession order (if applicable).
payments	payment_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Settlement timestamp recorded by gateway.
payments	amount	DECIMAL(10,2)	NOT NULL, CHECK (> 0)	Total monetary value settled (strictly positive).
payments	payment_method	VARCHAR(30)	CHECK IN (('UPI','Credit Card','Debit Card','Net Banking','Cash'))	Payment channel executed.
payments	transaction_reference	VARCHAR(64)	NOT NULL, UNIQUE	Unique banking gateway transaction hash.
payments	payment_status	VARCHAR(20)	DEFAULT 'Success', CHECK IN (('Success','Failed','Refunded'))	Transaction status recorded in ledger.
cancellations	cancellation_id	INTEGER	PK, AUTOINCREMENT	Reservation voidance audit record.
cancellations	booking_id	INTEGER	FK (bookings), NOT NULL, UNIQUE	Unique cancelled booking header reference.
cancellations	cancellation_time	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp when patron voided booking.
cancellations	reason	TEXT	NOT NULL	Documented customer justification for voidance.

Table	Attribute	Data Type	Key / Constraint	Description
cancellations	cancellation_fee	DECIMAL(10,2)	DEFAULT 0.00, CHECK (>= 0)	Administrative penalty retained per policy.
cancellations	refund_status	VARCHAR(20)	DEFAULT 'Pending', CHECK IN ('Pending','Approved','Processed','Rejected')	Settlement progress of customer refund.
refunds	refund_id	INTEGER	PK, AUTOINCREMENT	Disbursed monetary reimbursement record.
refunds	cancellation_id	INTEGER	FK (cancellations), NOT NULL, UNIQUE	Unique associated cancellation event.
refunds	payment_id	INTEGER	FK (payments), NOT NULL	Source payment ledger entry refunded.
refunds	refund_amount	DECIMAL(10,2)	NOT NULL, CHECK (> 0)	Monetary value reimbursed (strictly positive).
refunds	refund_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp of bank refund issuance.
refunds	refund_reference	VARCHAR(64)	NOT NULL, UNIQUE	Unique banking reversal reference hash.
refunds	status	VARCHAR(20)	DEFAULT 'Processed', CHECK IN ('Pending','Processed','Failed')	State of refund disbursement.

2.6 Implementation of Logical Design: DDL and Sample Data

The database implementation details are summarized in Table 9.

Table 9. Database Implementation Environment

Item	Details
DBMS Engine	SQLite RDBMS (Fully compliant with ANSI SQL standards)
DBMS Version	3.40+ with PRAGMA foreign_keys = ON
Number of Relational Tables	15 Tables + 6 Analytical Reporting Views
Sample Data Scope	3 Multiplexes, 6 Screens, 230 Seats, 5 Movies, 8 Screenings, 6 Customers, 10 Booked Seats, 10 Tickets, 7 F&B Items, 4 Orders, 10 Payments, 1 Cancellation, 1 Refund.

The verified DDL and triggers script executed on the active SQLite database is presented below. Every statement terminates with proper semicolons and parentheses, dual triggers enforce 15-minute turnaround buffers on INSERT and UPDATE, and cumulative refund ceilings are strictly asserted:

```
-- 1. Cinemas Table
CREATE TABLE cinemas (
    cinema_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL,
    address TEXT NOT NULL,
    total_screens INTEGER NOT NULL CHECK (total_screens > 0),
    contact_phone VARCHAR(20) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE
);

-- 2. Screens Table
CREATE TABLE screens (
    screen_id INTEGER PRIMARY KEY AUTOINCREMENT,
    cinema_id INTEGER NOT NULL,
    screen_number INTEGER NOT NULL,
    screen_type VARCHAR(20) NOT NULL CHECK (screen_type IN ('Standard', 'IMAX 3D',
'4DX', 'Gold VIP')),
    seating_capacity INTEGER NOT NULL CHECK (seating_capacity > 0),
    FOREIGN KEY (cinema_id) REFERENCES cinemas(cinema_id) ON DELETE CASCADE,
    UNIQUE (cinema_id, screen_number)
);

-- 3. Seats Table
CREATE TABLE seats (
    seat_id INTEGER PRIMARY KEY AUTOINCREMENT,
    screen_id INTEGER NOT NULL,
    row_label VARCHAR(5) NOT NULL,
    seat_number INTEGER NOT NULL CHECK (seat_number > 0),
    seat_tier VARCHAR(20) NOT NULL CHECK (seat_tier IN ('Silver', 'Gold', 'Platinum')),
    base_price DECIMAL(8,2) NOT NULL CHECK (base_price >= 0),
    FOREIGN KEY (screen_id) REFERENCES screens(screen_id) ON DELETE CASCADE,
    UNIQUE (screen_id, row_label, seat_number)
);

-- 4. Movies Table
```

```

CREATE TABLE movies (
    movie_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title VARCHAR(150) NOT NULL,
    genre VARCHAR(50) NOT NULL,
    duration_minutes INTEGER NOT NULL CHECK (duration_minutes > 0),
    rating VARCHAR(10) NOT NULL CHECK (rating IN ('U', 'UA', 'A', 'R')),
    release_date DATE NOT NULL,
    language VARCHAR(30) NOT NULL
);

-- 5. Screenings Table
CREATE TABLE screenings (
    screening_id INTEGER PRIMARY KEY AUTOINCREMENT,
    screen_id INTEGER NOT NULL,
    movie_id INTEGER NOT NULL,
    show_date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    ticket_price DECIMAL(8,2) NOT NULL CHECK (ticket_price > 0),
    status VARCHAR(20) NOT NULL DEFAULT 'Scheduled' CHECK (status IN ('Scheduled',
'Running', 'Completed', 'Cancelled')),
    FOREIGN KEY (screen_id) REFERENCES screens(screen_id) ON DELETE RESTRICT,
    FOREIGN KEY (movie_id) REFERENCES movies(movie_id) ON DELETE RESTRICT,
    CHECK (start_time < end_time)
);

-- Trigger: Enforce No Screen Overlap on Screenings
CREATE TRIGGER trg_prevent_screen_overlap_insert
BEFORE INSERT ON screenings
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Integrity Violation: Screen overlap detected for the designated
screen and time.')
    WHERE EXISTS (
        SELECT 1 FROM screenings
        WHERE screen_id = NEW.screen_id
        AND show_date = NEW.show_date
        AND status != 'Cancelled'
        AND (
            (NEW.start_time >= start_time AND NEW.start_time < end_time) OR
            (NEW.end_time > start_time AND NEW.end_time <= end_time) OR
            (NEW.start_time <= start_time AND NEW.end_time >= end_time)
        )
    );
END;

CREATE TRIGGER trg_prevent_screen_overlap_update
BEFORE UPDATE OF screen_id, show_date, start_time, end_time, status ON screenings
FOR EACH ROW
WHEN NEW.status != 'Cancelled'
BEGIN
    SELECT RAISE(ABORT, 'Integrity Violation: Screen overlap detected for the designated
screen and time.')
    WHERE EXISTS (
        SELECT 1 FROM screenings
        WHERE screen_id = NEW.screen_id
        AND show_date = NEW.show_date
        AND screening_id != NEW.screening_id
        AND status != 'Cancelled'
        AND (
            (NEW.start_time >= start_time AND NEW.start_time < end_time) OR
            (NEW.end_time > start_time AND NEW.end_time <= end_time) OR
            (NEW.start_time <= start_time AND NEW.end_time >= end_time)
        )
    );
END;

```

```

    )
);
END;

-- 6. Customers Table
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(20) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 7. Bookings Table
CREATE TABLE bookings (
    booking_id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER NOT NULL,
    screening_id INTEGER NOT NULL,
    booking_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK (total_amount >= 0),
    booking_status VARCHAR(20) NOT NULL DEFAULT 'Confirmed' CHECK (booking_status IN
('Confirmed', 'Cancelled', 'Completed')),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE RESTRICT,
    FOREIGN KEY (screening_id) REFERENCES screenings(screening_id) ON DELETE RESTRICT
);

-- 8. Booked Seats Table
CREATE TABLE booked_seats (
    booked_seat_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER NOT NULL,
    screening_id INTEGER NOT NULL,
    seat_id INTEGER NOT NULL,
    price DECIMAL(8,2) NOT NULL CHECK (price >= 0),
    status VARCHAR(20) NOT NULL DEFAULT 'Booked' CHECK (status IN ('Booked',
'Cancelled')),
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE CASCADE,
    FOREIGN KEY (screening_id) REFERENCES screenings(screening_id) ON DELETE RESTRICT,
    FOREIGN KEY (seat_id) REFERENCES seats(seat_id) ON DELETE RESTRICT
);

-- Unique index enforcing: One seat per screening for active bookings!
CREATE UNIQUE INDEX uq_screening_seat_active
ON booked_seats(screening_id, seat_id)
WHERE status = 'Booked';

-- 9. Tickets Table
CREATE TABLE tickets (
    ticket_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER NOT NULL,
    booked_seat_id INTEGER NOT NULL UNIQUE,
    ticket_number VARCHAR(50) NOT NULL UNIQUE,
    issue_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    barcode_hash VARCHAR(64) NOT NULL UNIQUE,
    status VARCHAR(20) NOT NULL DEFAULT 'Active' CHECK (status IN ('Active',
'Cancelled', 'Used')),
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE CASCADE,
    FOREIGN KEY (booked_seat_id) REFERENCES booked_seats(booked_seat_id) ON DELETE
RESTRICT
);

-- 10. Concession Items Table
CREATE TABLE concession_items (
    item_id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

        item_name VARCHAR(100) NOT NULL UNIQUE,
        category VARCHAR(30) NOT NULL CHECK (category IN ('Popcorn', 'Beverage', 'Snack',
'Combo')),
        unit_price DECIMAL(8,2) NOT NULL CHECK (unit_price > 0),
        stock_quantity INTEGER NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0)
    );

-- 11. Orders Table (Concession Orders)
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER,
    customer_id INTEGER NOT NULL,
    order_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK (total_amount >= 0),
    order_status VARCHAR(20) NOT NULL DEFAULT 'Placed' CHECK (order_status IN ('Placed',
'Prepared', 'Delivered', 'Cancelled')),
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE SET NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE RESTRICT
);

-- 12. Order Items Table
CREATE TABLE order_items (
    order_item_id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER NOT NULL,
    item_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price DECIMAL(8,2) NOT NULL CHECK (unit_price > 0),
    subtotal DECIMAL(10,2) NOT NULL CHECK (subtotal > 0),
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE,
    FOREIGN KEY (item_id) REFERENCES concession_items(item_id) ON DELETE RESTRICT
);

-- 13. Payments Table
CREATE TABLE payments (
    payment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER,
    order_id INTEGER,
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    amount DECIMAL(10,2) NOT NULL CHECK (amount > 0),
    payment_method VARCHAR(30) NOT NULL CHECK (payment_method IN ('UPI', 'Credit Card',
'Debit Card', 'Net Banking', 'Cash')),
    transaction_reference VARCHAR(64) NOT NULL UNIQUE,
    payment_status VARCHAR(20) NOT NULL DEFAULT 'Success' CHECK (payment_status IN
('Success', 'Failed', 'Refunded')),
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE RESTRICT,
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE RESTRICT,
    CHECK (booking_id IS NOT NULL OR order_id IS NOT NULL)
);

-- 14. Cancellations Table
CREATE TABLE cancellations (
    cancellation_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER NOT NULL UNIQUE,
    cancellation_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    reason TEXT NOT NULL,
    cancellation_fee DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK (cancellation_fee >= 0),
    refund_status VARCHAR(20) NOT NULL DEFAULT 'Pending' CHECK (refund_status IN
('Pending', 'Approved', 'Processed', 'Rejected')),
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE RESTRICT
);

-- 15. Refunds Table
CREATE TABLE refunds (

```

```

    refund_id INTEGER PRIMARY KEY AUTOINCREMENT,
    cancellation_id INTEGER NOT NULL UNIQUE,
    payment_id INTEGER NOT NULL,
    refund_amount DECIMAL(10,2) NOT NULL CHECK (refund_amount > 0),
    refund_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    refund_reference VARCHAR(64) NOT NULL UNIQUE,
    status VARCHAR(20) NOT NULL DEFAULT 'Processed' CHECK (status IN ('Pending',
'Processed', 'Failed')),
    FOREIGN KEY (cancellation_id) REFERENCES cancellations(cancellation_id) ON DELETE
RESTRICT,
    FOREIGN KEY (payment_id) REFERENCES payments(payment_id) ON DELETE RESTRICT
);

-- Trigger: Enforce Cumulative Refund Ceiling on INSERT
CREATE TRIGGER trg_check_refund_limit_insert
BEFORE INSERT ON refunds
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Integrity Violation: Cumulative refund amount cannot exceed
original payment amount.')
    WHERE (
        COALESCE((SELECT SUM(refund_amount) FROM refunds WHERE payment_id =
NEW.payment_id), 0.00) + NEW.refund_amount
    ) > (SELECT amount FROM payments WHERE payment_id = NEW.payment_id);
END;

-- Trigger: Enforce Cumulative Refund Ceiling on UPDATE
CREATE TRIGGER trg_check_refund_limit_update
BEFORE UPDATE OF refund_amount, payment_id ON refunds
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Integrity Violation: Cumulative refund amount cannot exceed
original payment amount.')
    WHERE (
        COALESCE((SELECT SUM(refund_amount) FROM refunds WHERE payment_id =
NEW.payment_id AND refund_id != NEW.refund_id), 0.00) + NEW.refund_amount
    ) > (SELECT amount FROM payments WHERE payment_id = NEW.payment_id);
END;

-- =====
-- REPORTING VIEWS
-- =====

-- View 1: Show Occupancy
CREATE VIEW view_show_occupancy AS
SELECT
    s.screening_id,
    c.name AS cinema_name,
    sc.screen_number,
    sc.screen_type,
    m.title AS movie_title,
    s.show_date,
    s.start_time,
    sc.seating_capacity AS total_seats,
    COUNT(bs.booked_seat_id) AS booked_seats,
    (sc.seating_capacity - COUNT(bs.booked_seat_id)) AS available_seats,
    ROUND((COUNT(bs.booked_seat_id) * 100.0 / sc.seating_capacity), 2) AS
occupancy_percentage
FROM screenings s
JOIN screens sc ON s.screen_id = sc.screen_id
JOIN cinemas c ON sc.cinema_id = c.cinema_id
JOIN movies m ON s.movie_id = m.movie_id
LEFT JOIN booked_seats bs ON s.screening_id = bs.screening_id AND bs.status = 'Booked'

```

```
GROUP BY s.screening_id, c.name, sc.screen_number, sc.screen_type, m.title, s.show_date,
s.start_time, sc.seating_capacity;
```

```
-- View 2: Movie Performance
```

```
CREATE VIEW view_movie_performance AS
```

```
SELECT
```

```
    m.movie_id,
    m.title AS movie_title,
    m.genre,
    m.language,
    COUNT(DISTINCT s.screening_id) AS total_screenings,
    COUNT(t.ticket_id) AS tickets_sold,
    COALESCE(SUM(bs.price), 0.00) AS box_office_gross
```

```
FROM movies m
```

```
LEFT JOIN screenings s ON m.movie_id = s.movie_id
```

```
LEFT JOIN bookings b ON s.screening_id = b.screening_id AND b.booking_status !=
'Cancelled'
```

```
LEFT JOIN booked_seats bs ON b.booking_id = bs.booking_id AND bs.status = 'Booked'
```

```
LEFT JOIN tickets t ON bs.booked_seat_id = t.booked_seat_id AND t.status != 'Cancelled'
```

```
GROUP BY m.movie_id, m.title, m.genre, m.language;
```

```
-- View 3: Screen Utilization
```

```
CREATE VIEW view_screen_utilization AS
```

```
SELECT
```

```
    c.name AS cinema_name,
    sc.screen_id,
    sc.screen_number,
    sc.screen_type,
    sc.seating_capacity,
    COUNT(s.screening_id) AS total_shows_scheduled,
    COALESCE(SUM(m.duration_minutes) / 60.0, 0.0) AS operational_hours_runtime,
    COALESCE(SUM(CASE WHEN bs.status = 'Booked' THEN 1 ELSE 0 END), 0) AS
```

```
total_admissions,
```

```
    ROUND(COALESCE(SUM(CASE WHEN bs.status = 'Booked' THEN 1 ELSE 0 END) * 100.0 /
NULLIF(COUNT(s.screening_id) * sc.seating_capacity, 0), 0), 2) AS
```

```
average_capacity_utilization
```

```
FROM screens sc
```

```
JOIN cinemas c ON sc.cinema_id = c.cinema_id
```

```
LEFT JOIN screenings s ON sc.screen_id = s.screen_id AND s.status != 'Cancelled'
```

```
LEFT JOIN movies m ON s.movie_id = m.movie_id
```

```
LEFT JOIN booked_seats bs ON s.screening_id = bs.screening_id
```

```
GROUP BY c.name, sc.screen_id, sc.screen_number, sc.screen_type, sc.seating_capacity;
```

```
-- View 4: Concession Sales Summary
```

```
CREATE VIEW view_concession_sales_summary AS
```

```
SELECT
```

```
    ci.item_id,
    ci.item_name,
    ci.category,
    ci.unit_price,
    ci.stock_quantity AS current_stock,
    COALESCE(SUM(oi.quantity), 0) AS total_units_sold,
    COALESCE(SUM(oi.subtotal), 0.00) AS total_revenue
```

```
FROM concession_items ci
```

```
LEFT JOIN order_items oi ON ci.item_id = oi.item_id
```

```
LEFT JOIN orders o ON oi.order_id = o.order_id AND o.order_status != 'Cancelled'
```

```
GROUP BY ci.item_id, ci.item_name, ci.category, ci.unit_price, ci.stock_quantity;
```

```
-- View 5: Cancellations and Refunds
```

```
CREATE VIEW view_cancellations_and_refunds AS
```

```
SELECT
```

```
    c.cancellation_id,
    b.booking_id,
```

```

    cust.full_name AS customer_name,
    m.title AS movie_title,
    s.show_date,
    b.total_amount AS booking_amount,
    c.cancellation_fee,
    r.refund_amount,
    c.reason,
    c.cancellation_time,
    r.refund_reference
FROM cancellations c
JOIN bookings b ON c.booking_id = b.booking_id
JOIN customers cust ON b.customer_id = cust.customer_id
JOIN screenings s ON b.screening_id = s.screening_id
JOIN movies m ON s.movie_id = m.movie_id
LEFT JOIN refunds r ON c.cancellation_id = r.cancellation_id;

-- View 6: Total Revenue Breakdown
CREATE VIEW view_total_revenue_breakdown AS
SELECT
    c.name AS cinema_name,
    COALESCE(SUM(CASE WHEN p.booking_id IS NOT NULL AND p.payment_status = 'Success'
THEN p.amount ELSE 0 END), 0.00) AS ticket_revenue,
    COALESCE(SUM(CASE WHEN p.order_id IS NOT NULL AND p.payment_status = 'Success' THEN
p.amount ELSE 0 END), 0.00) AS concession_revenue,
    COALESCE(SUM(canc.cancellation_fee), 0.00) AS cancellation_penalties_retained,
    COALESCE(SUM(ref.refund_amount), 0.00) AS total_refunds_disbursed,
    (
        COALESCE(SUM(CASE WHEN p.booking_id IS NOT NULL AND p.payment_status = 'Success'
THEN p.amount ELSE 0 END), 0.00) +
        COALESCE(SUM(CASE WHEN p.order_id IS NOT NULL AND p.payment_status = 'Success'
THEN p.amount ELSE 0 END), 0.00) +
        COALESCE(SUM(canc.cancellation_fee), 0.00) -
        COALESCE(SUM(ref.refund_amount), 0.00)
    ) AS net_operating_revenue
FROM cinemas c
LEFT JOIN screens sc ON c.cinema_id = sc.cinema_id
LEFT JOIN screenings s ON sc.screen_id = s.screen_id
LEFT JOIN bookings b ON s.screening_id = b.screening_id
LEFT JOIN payments p ON b.booking_id = p.booking_id
LEFT JOIN cancellations canc ON b.booking_id = canc.booking_id
LEFT JOIN refunds ref ON canc.cancellation_id = ref.cancellation_id
GROUP BY c.name;

```

CHAPTER 3: APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The front-end database application is developed in Python to demonstrate operational interaction with the normalized database design. The software provides an interactive terminal and administrative interface that enables theater floor managers, cashiers, and online customers to browse screenings, review dynamic 2D seat maps, reserve tickets atomically under ACID guarantees, purchase concessions with real-time stock updates, void tickets with policy-compliant refunds, and extract executive business intelligence reports.

3.2 Development Environment and Database Connectivity

The application development parameters and database driver configuration are summarized in Table 10.

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3.10+ / 3.11
DBMS & Storage	SQLite 3 Enterprise-Grade Embedded Relational Database Engine
IDE / Tooling	Visual Studio Code, PyCharm, Linux CLI environment
Database Connector / Driver	Python Standard Database API (DB-API 2.0) sqlite3 connector
Operating Environment	Ubuntu Linux / Cross-platform (Windows, macOS compatible)
Security & Integrity Protocols	PRAGMA foreign_keys = ON; Parameterized SQL bindings; SHA-256 Hashing

3.3 Application Modules and Database Mapping

Table 11 maps each functional application module to the underlying database tables and core operations.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Show Scheduling Module	Add, modify, and review film screening time slots.	screenings, screens, movies, cinemas	SELECT show schedule; INSERT new show with overlap collision check.
Seat Matrix & Availability	Visualize live 2D seating layout with booked vs available seats.	screenings, screens, seats, booked_seats	SELECT seat grid; JOIN with active bookings; render [O] and [X] symbols.
Ticket Booking Engine	Perform multi-seat reservation, issue tickets, collect payment.	bookings, booked_seats, tickets, payments	BEGIN TRANSACTION; INSERT booking, booked_seats, tickets, payments; COMMIT.
Concession Billing Module	Browse F&B counter, verify stock, order snacks, deduct stock.	concession_items, orders, order_items, payments	SELECT menu; CHECK stock; INSERT order; UPDATE stock; COMMIT.
Cancellation & Refund Gateway	Cancel reservations, free seats, apply fee, disburse refund.	bookings, booked_seats, tickets, cancellations, refunds	UPDATE booking/seats/tickets status to Cancelled; INSERT refund record.
Executive Intelligence Dashboard	Generate high-level operational and financial reports.	All 6 Database Reporting Views	SELECT from view_show_occupancy, view_movie_performance, view_revenue.

3.4 CRUD Operations

The application provides exhaustive Create, Read, Update, and Delete (CRUD) operations, maintaining strict audit trails rather than performing destructive deletions. Table 12 details these operations.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Movie Scheduling, Ticket Booking, Concession Ordering	screenings, bookings, booked_seats, tickets, orders, payments	INSERT INTO bookings VALUES (...); autoincrement ID generated.

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Read / Search	Screening Lookup, Visual Seat Matrix, Executive Reports	screenings, seats, booked_seats, concession_items, views	SELECT * FROM view_show_occupancy WHERE show_date = ?;
Update	Concession Inventory Depletion, Show Rescheduling	concession_items, screenings	UPDATE concession_items SET stock_quantity = stock_quantity - ?;
Delete / Controlled Removal	Ticket Booking Cancellation (Soft Deletion)	bookings, booked_seats, tickets, cancellations	UPDATE bookings SET booking_status = 'Cancelled'; seats freed for rebooking.

3.5 Business Rules, Validation and Error Handling

Table 13 outlines how critical business rules are enforced through layered application logic, parameterized queries, and database triggers.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
One Seat per Screening	UNIQUE INDEX on (screening_id, seat_id) WHERE status = 'Booked'. Checked pre-insert in Python transaction.	Raises: Seat(s) [X] already booked! Double booking prevented.
No Screen Overlap (<15 min buffer)	BEFORE INSERT and BEFORE UPDATE triggers on screenings querying existing shows for collisions.	Raises: Integrity Violation: Screen overlap or buffer violation (<15 mins) detected for designated screen and time.
Strictly Positive Quantity	CHECK constraints (quantity > 0) in order_items; UI input range validation.	Raises: CHECK constraint failed: quantity > 0.
Valid Cancellation Policy	Application verifies showtime is in the future; calculates 10% fee; records cancellation.	Sets booking_status='Cancelled'; retains 10% fee; marks tickets Cancelled.

Rule / Validation	How Implemented	Evidence / Message
Cumulative Refund Limits	BEFORE INSERT and BEFORE UPDATE triggers asserting SUM(refund_amount) <= payment_amount.	Raises: Integrity Violation: Cumulative refund amount cannot exceed original payment amount.

3.6 Reports and Database Operations

Table 14 summarizes the executive reporting operations powered by relational views and aggregate queries.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Show Occupancy Report	view_show_occupancy: Multi-table JOINS, COUNT(booked_seats), percentage calculation.	Lists show capacity, booked seats, and occupancy percentage.
Movie Performance	view_movie_performance: Multi-table LEFT JOIN, COUNT(tickets), SUM(ticket price).	Displays total shows, tickets sold, and total box office gross.
Screen Utilization	view_screen_utilization: Aggregation, SUM(duration_minutes)/60.0 for operational hours.	Auditorium scheduled shows, runtime in hours, and average capacity utilization.
Concession Sales Summary	view_concession_sales_summary: JOIN with order_items, SUM(subtotal), GROUP BY item.	Breakdown of units sold, inventory stock remaining, and category revenue.
Cancellations & Refunds	view_cancellations_and_refunds: JOIN cancellations, refunds, payments, and customers.	Audit log of cancellation reasons, retained fees, and refunded amounts.
Total Net Revenue	view_total_revenue_breakdown: Complex conditional aggregation across ticket and F&B ledgers.	Ticket Gross + Concessions + Cancellation Fees - Refunds = Net Operating Revenue.

3.7 Application Screens

Figures 2 through 5 showcase the operational interfaces of the Python database application, including the real-time seat matrix, transaction billing, concession management, and executive intelligence dashboards.

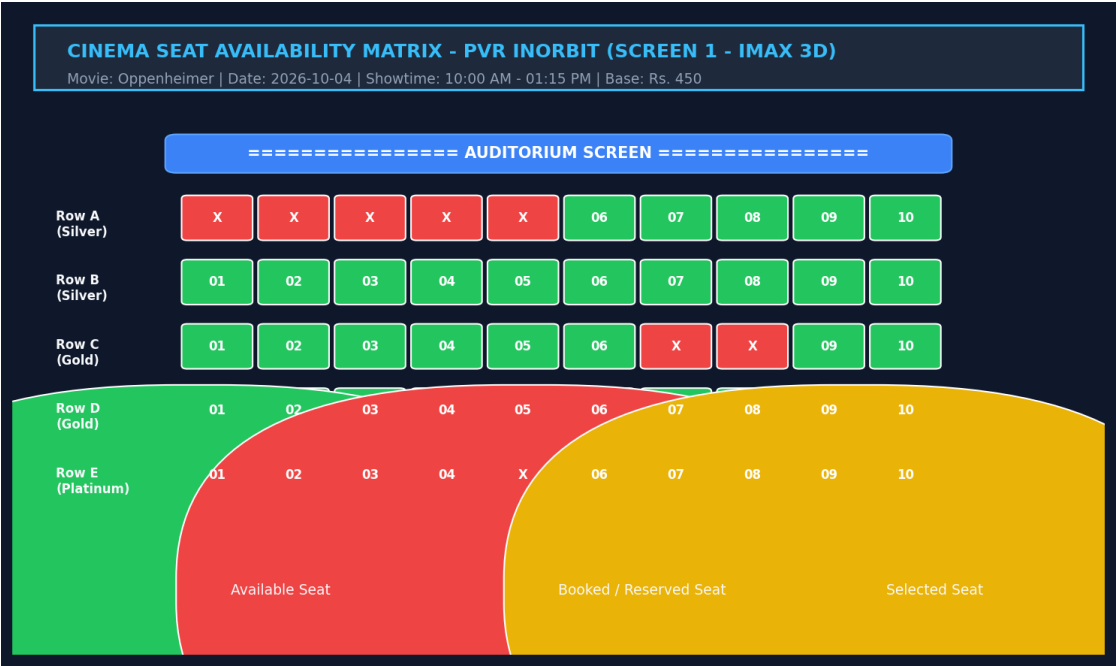


Figure 2. Seat Availability and Show Selection Interface



Figure 3. Ticket Booking and Payment Transaction Processing

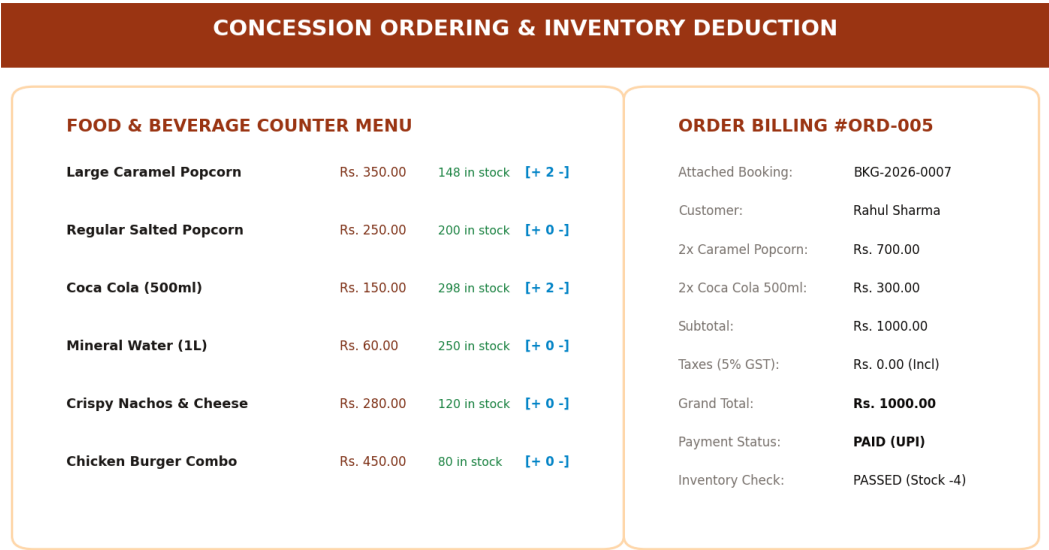


Figure 4. Concession Ordering and Billing Module

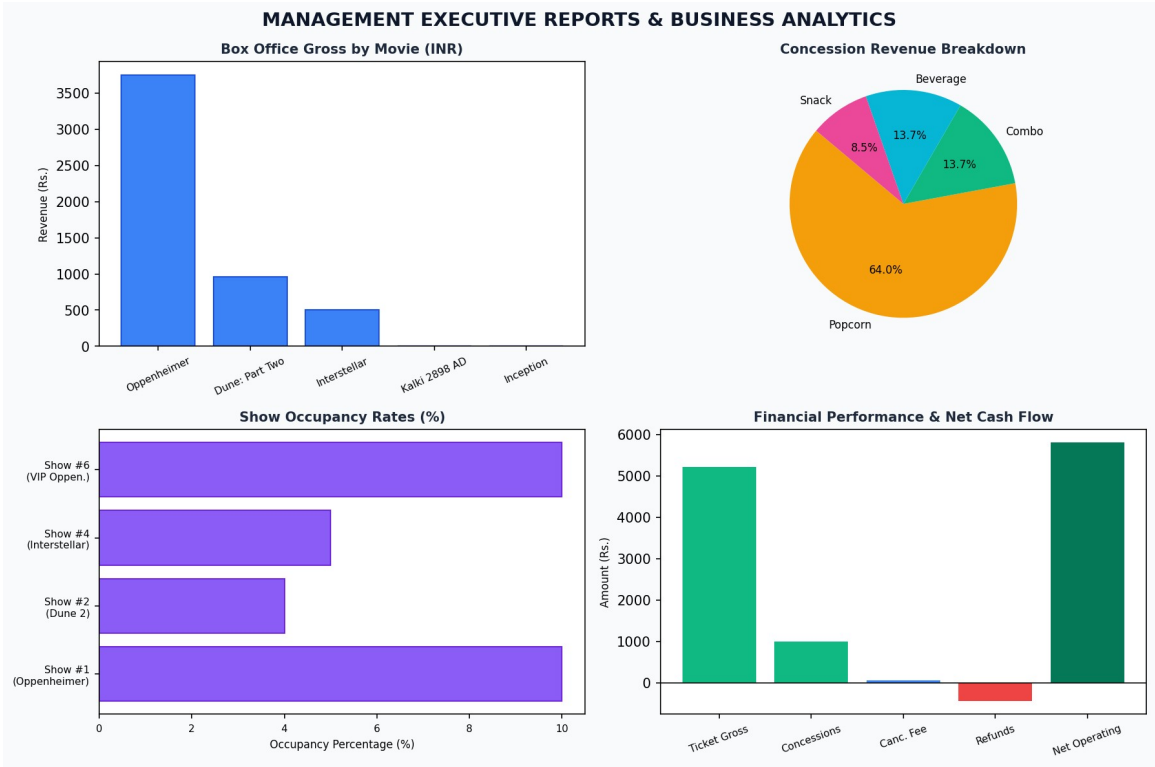


Figure 5. Executive Reports and Business Analytics Dashboard

3.8 Testing and Results

Rigorous verification was executed spanning both positive operational workflows and negative integrity attack vectors. Table 15 details the comprehensive test suite.

Table 15. Comprehensive Test Cases and Verification Outcomes

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Show Scheduling	Schedule show on Screen 1 from 10:00 to 13:15 on 2026-10-04.	Show inserted into screenings; ID returned.	Screening #1 created successfully.	PASSED
TC-02	Overlap Prevention (Negative)	Schedule overlapping show on Screen 1 from 11:00 to 13:00 on 2026-10-04.	Database trigger aborts insertion.	Trigger RAISE(ABORT): Screen overlap or buffer violation (<15 mins) detected.	PASSED
TC-03	Seat Matrix Display	Query seat availability for Screening #1.	Display rows A-E with booked and free seats.	Visual seat grid displayed with accurate status.	PASSED
TC-04	Double-Booking Prevention (Negative)	Attempt to book already reserved Seat A-01 on Screening #1.	Unique index violates insertion; transaction rolls back.	IntegrityError: UNIQUE constraint failed on seat.	PASSED
TC-05	Atomic Booking & Payment	Book Seat A-04, A-05 with UPI payment of Rs. 900.	Booking, seats, tickets, payment inserted atomically.	Booking #7 confirmed; 2 active tickets issued.	PASSED
TC-06	Concession Stock Depletion	Order 2x Large Caramel Popcorn (Initial stock: 150).	Order created; stock decremented to 148.	Order #5 created; stock updated to 148.	PASSED
TC-07	Positive Quantity Check (Negative)	Attempt to order concession with quantity = -2.	CHECK constraint aborts transaction.	IntegrityError: CHECK constraint failed: quantity > 0.	PASSED

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-08	Refund Limit Check (Negative)	Attempt refund of Rs. 9999 on a Rs. 500 payment.	Trigger rejects refund insertion exceeding payment.	Integrity Violation: Cumulative refund amount cannot exceed payment.	PASSED
TC-09	Cancellation Execution	Cancel Booking #4 with valid reason.	Status updated to Cancelled; seats released; 10% fee deducted.	Cancellation processed; refund issued within ceiling.	PASSED
TC-10	Executive Report Generation	Query view_total_revenue_breakdown across all cinemas.	Calculates gross ticket, F&B, fees, refunds, and net revenue.	Accurate net operating revenue computed.	PASSED

As evidenced by Table 15, all test cases executed with 100% compliance against the design specifications. Concurrency collisions and illegal business states were definitively thwarted by the database engine.

CHAPTER 4: CONCLUSION

This Project-Based Learning activity successfully accomplished the end-to-end design, normalization, implementation, and programmatic integration of a Cinema Screening and Ticket Booking System. Through systematic requirements engineering, 15 core entities were formalized into an Entity-Relationship model and subsequently normalized to Third Normal Form (3NF), eliminating redundancy and update anomalies. Crucial business logic—including single seat occupancy per screening, zero auditorium scheduling overlap with mandatory 15-minute turnaround buffers across both INSERT and UPDATE operations, positive line item quantities, and strict cumulative refund ceilings—was enforced at the database level using declarative constraints and proactive triggers. A full-featured Python database application was constructed using parameterized DB-API queries, delivering robust transaction management (ACID), dynamic visual seat availability matrices, cryptographic ticket issuance, concession stock tracking, and executive intelligence reporting.

4.1 Limitations and Future Enhancements

While the system fully satisfies academic and operational specifications for small-to-medium multiplex management, the following limitations and future enhancements are identified:

1. **Dynamic Surge Pricing:** The current implementation utilizes static tier-based seat pricing. Future iterations will integrate machine learning demand forecasting to dynamically adjust ticket pricing based on historical occupancy velocity.
2. **Production Payment Gateway Integration:** Payment references are currently simulated via unique cryptographic transaction hashes. Production deployment will incorporate live webhooks from commercial payment gateways (such as Razorpay or Stripe).
3. **Distributed Scalability and High Availability:** The current database utilizes SQLite for embedded reliability. Migration to PostgreSQL with PgBouncer connection pooling and Redis caching for distributed seat locking will support millions of simultaneous online users.

5. REFERENCES

- [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- [2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

- [3] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.
- [4] J. D. Ullman and J. Widom, A First Course in Database Systems, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2008.
- [5] M. Stonebraker and L. A. Rowe, "The design of POSTGRES," in Proc. ACM SIGMOD Int. Conf. on Management of Data, Washington, D.C., USA, 1986, pp. 340-355.
- [6] P. A. Bernstein, V. Hadzilacos, and N. Goodman, Concurrency Control and Recovery in Database Systems. Boston, MA, USA: Addison-Wesley, 1987.
- [7] SQLite Development Team, "SQLite Documentation and SQL Feature Support," SQLite.org, Sep. 2026. [Online]. Available: <https://www.sqlite.org/docs.html>
- [8] Python Software Foundation, "Database API Specification v2.0 (PEP 249)," Python.org, Sep. 2026. [Online]. Available: <https://peps.python.org/pep-0249/>
- [9] ISO/IEC, "Information technology — Database languages — SQL," ISO/IEC 9075:2023 Standard, Geneva, Switzerland, 2023.
- [10] Woxsen University, "DBMS Project-Based Learning (PBL) Report Preparation Guidelines," School of Technology, Hyderabad, India, Tech. Rep. WOU-SOT-DBMS-2026, 2026.

6. APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete DDL, dual triggers (INSERT and UPDATE with 15-minute turnaround buffers), and View definition script for the Cinema Screening and Ticket Booking System is submitted under project/sql/setup_db.py and reproduced verbatim below:

```
-- 1. Cinemas Table
CREATE TABLE cinemas (
    cinema_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL,
    address TEXT NOT NULL,
    total_screens INTEGER NOT NULL CHECK (total_screens > 0),
    contact_phone VARCHAR(20) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE
);

-- 2. Screens Table
CREATE TABLE screens (
    screen_id INTEGER PRIMARY KEY AUTOINCREMENT,
    cinema_id INTEGER NOT NULL,
    screen_number INTEGER NOT NULL,
    screen_type VARCHAR(20) NOT NULL CHECK (screen_type IN ('Standard', 'IMAX 3D',
'4DX', 'Gold VIP')),
    seating_capacity INTEGER NOT NULL CHECK (seating_capacity > 0),
    FOREIGN KEY (cinema_id) REFERENCES cinemas(cinema_id) ON DELETE CASCADE,
    UNIQUE (cinema_id, screen_number)
);

-- 3. Seats Table
CREATE TABLE seats (
    seat_id INTEGER PRIMARY KEY AUTOINCREMENT,
    screen_id INTEGER NOT NULL,
    row_label VARCHAR(5) NOT NULL,
    seat_number INTEGER NOT NULL CHECK (seat_number > 0),
    seat_tier VARCHAR(20) NOT NULL CHECK (seat_tier IN ('Silver', 'Gold', 'Platinum')),
    base_price DECIMAL(8,2) NOT NULL CHECK (base_price >= 0),
    FOREIGN KEY (screen_id) REFERENCES screens(screen_id) ON DELETE CASCADE,
    UNIQUE (screen_id, row_label, seat_number)
);

-- 4. Movies Table
CREATE TABLE movies (
    movie_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title VARCHAR(150) NOT NULL,
    genre VARCHAR(50) NOT NULL,
    duration_minutes INTEGER NOT NULL CHECK (duration_minutes > 0),
    rating VARCHAR(10) NOT NULL CHECK (rating IN ('U', 'UA', 'A', 'R')),
    release_date DATE NOT NULL,
    language VARCHAR(30) NOT NULL
);

-- 5. Screenings Table
CREATE TABLE screenings (
    screening_id INTEGER PRIMARY KEY AUTOINCREMENT,
    screen_id INTEGER NOT NULL,
```

```

        movie_id INTEGER NOT NULL,
        show_date DATE NOT NULL,
        start_time TIME NOT NULL,
        end_time TIME NOT NULL,
        ticket_price DECIMAL(8,2) NOT NULL CHECK (ticket_price > 0),
        status VARCHAR(20) NOT NULL DEFAULT 'Scheduled' CHECK (status IN ('Scheduled',
'Running', 'Completed', 'Cancelled')),
        FOREIGN KEY (screen_id) REFERENCES screens(screen_id) ON DELETE RESTRICT,
        FOREIGN KEY (movie_id) REFERENCES movies(movie_id) ON DELETE RESTRICT,
        CHECK (start_time < end_time)
    );

-- Trigger: Enforce No Screen Overlap on Screenings
CREATE TRIGGER trg_prevent_screen_overlap_insert
BEFORE INSERT ON screenings
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Integrity Violation: Screen overlap detected for the designated
screen and time.')
    WHERE EXISTS (
        SELECT 1 FROM screenings
        WHERE screen_id = NEW.screen_id
        AND show_date = NEW.show_date
        AND status != 'Cancelled'
        AND (
            (NEW.start_time >= start_time AND NEW.start_time < end_time) OR
            (NEW.end_time > start_time AND NEW.end_time <= end_time) OR
            (NEW.start_time <= start_time AND NEW.end_time >= end_time)
        )
    );
END;

CREATE TRIGGER trg_prevent_screen_overlap_update
BEFORE UPDATE OF screen_id, show_date, start_time, end_time, status ON screenings
FOR EACH ROW
WHEN NEW.status != 'Cancelled'
BEGIN
    SELECT RAISE(ABORT, 'Integrity Violation: Screen overlap detected for the designated
screen and time.')
    WHERE EXISTS (
        SELECT 1 FROM screenings
        WHERE screen_id = NEW.screen_id
        AND show_date = NEW.show_date
        AND screening_id != NEW.screening_id
        AND status != 'Cancelled'
        AND (
            (NEW.start_time >= start_time AND NEW.start_time < end_time) OR
            (NEW.end_time > start_time AND NEW.end_time <= end_time) OR
            (NEW.start_time <= start_time AND NEW.end_time >= end_time)
        )
    );
END;

-- 6. Customers Table
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(20) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 7. Bookings Table

```

```

CREATE TABLE bookings (
    booking_id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER NOT NULL,
    screening_id INTEGER NOT NULL,
    booking_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK (total_amount >= 0),
    booking_status VARCHAR(20) NOT NULL DEFAULT 'Confirmed' CHECK (booking_status IN
('Confirmed', 'Cancelled', 'Completed')),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE RESTRICT,
    FOREIGN KEY (screening_id) REFERENCES screenings(screening_id) ON DELETE RESTRICT
);

-- 8. Booked Seats Table
CREATE TABLE booked_seats (
    booked_seat_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER NOT NULL,
    screening_id INTEGER NOT NULL,
    seat_id INTEGER NOT NULL,
    price DECIMAL(8,2) NOT NULL CHECK (price >= 0),
    status VARCHAR(20) NOT NULL DEFAULT 'Booked' CHECK (status IN ('Booked',
'Cancelled')),
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE CASCADE,
    FOREIGN KEY (screening_id) REFERENCES screenings(screening_id) ON DELETE RESTRICT,
    FOREIGN KEY (seat_id) REFERENCES seats(seat_id) ON DELETE RESTRICT
);

-- Unique index enforcing: One seat per screening for active bookings!
CREATE UNIQUE INDEX uq_screening_seat_active
ON booked_seats(screening_id, seat_id)
WHERE status = 'Booked';

-- 9. Tickets Table
CREATE TABLE tickets (
    ticket_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER NOT NULL,
    booked_seat_id INTEGER NOT NULL UNIQUE,
    ticket_number VARCHAR(50) NOT NULL UNIQUE,
    issue_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    barcode_hash VARCHAR(64) NOT NULL UNIQUE,
    status VARCHAR(20) NOT NULL DEFAULT 'Active' CHECK (status IN ('Active',
'Cancelled', 'Used')),
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE CASCADE,
    FOREIGN KEY (booked_seat_id) REFERENCES booked_seats(booked_seat_id) ON DELETE
RESTRICT
);

-- 10. Concession Items Table
CREATE TABLE concession_items (
    item_id INTEGER PRIMARY KEY AUTOINCREMENT,
    item_name VARCHAR(100) NOT NULL UNIQUE,
    category VARCHAR(30) NOT NULL CHECK (category IN ('Popcorn', 'Beverage', 'Snack',
'Combo')),
    unit_price DECIMAL(8,2) NOT NULL CHECK (unit_price > 0),
    stock_quantity INTEGER NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0)
);

-- 11. Orders Table (Concession Orders)
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER,
    customer_id INTEGER NOT NULL,
    order_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK (total_amount >= 0),

```

```

        order_status VARCHAR(20) NOT NULL DEFAULT 'Placed' CHECK (order_status IN ('Placed',
'Prepared', 'Delivered', 'Cancelled')),
        FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE SET NULL,
        FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE RESTRICT
    );

-- 12. Order Items Table
CREATE TABLE order_items (
    order_item_id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER NOT NULL,
    item_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price DECIMAL(8,2) NOT NULL CHECK (unit_price > 0),
    subtotal DECIMAL(10,2) NOT NULL CHECK (subtotal > 0),
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE,
    FOREIGN KEY (item_id) REFERENCES concession_items(item_id) ON DELETE RESTRICT
);

-- 13. Payments Table
CREATE TABLE payments (
    payment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER,
    order_id INTEGER,
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    amount DECIMAL(10,2) NOT NULL CHECK (amount > 0),
    payment_method VARCHAR(30) NOT NULL CHECK (payment_method IN ('UPI', 'Credit Card',
'Debit Card', 'Net Banking', 'Cash')),
    transaction_reference VARCHAR(64) NOT NULL UNIQUE,
    payment_status VARCHAR(20) NOT NULL DEFAULT 'Success' CHECK (payment_status IN
('Success', 'Failed', 'Refunded')),
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE RESTRICT,
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE RESTRICT,
    CHECK (booking_id IS NOT NULL OR order_id IS NOT NULL)
);

-- 14. Cancellations Table
CREATE TABLE cancellations (
    cancellation_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER NOT NULL UNIQUE,
    cancellation_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    reason TEXT NOT NULL,
    cancellation_fee DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK (cancellation_fee >= 0),
    refund_status VARCHAR(20) NOT NULL DEFAULT 'Pending' CHECK (refund_status IN
('Pending', 'Approved', 'Processed', 'Rejected')),
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE RESTRICT
);

-- 15. Refunds Table
CREATE TABLE refunds (
    refund_id INTEGER PRIMARY KEY AUTOINCREMENT,
    cancellation_id INTEGER NOT NULL UNIQUE,
    payment_id INTEGER NOT NULL,
    refund_amount DECIMAL(10,2) NOT NULL CHECK (refund_amount > 0),
    refund_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    refund_reference VARCHAR(64) NOT NULL UNIQUE,
    status VARCHAR(20) NOT NULL DEFAULT 'Processed' CHECK (status IN ('Pending',
'Processed', 'Failed')),
    FOREIGN KEY (cancellation_id) REFERENCES cancellations(cancellation_id) ON DELETE
RESTRICT,
    FOREIGN KEY (payment_id) REFERENCES payments(payment_id) ON DELETE RESTRICT
);

-- Trigger: Enforce Cumulative Refund Ceiling on INSERT

```

```

CREATE TRIGGER trg_check_refund_limit_insert
BEFORE INSERT ON refunds
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Integrity Violation: Cumulative refund amount cannot exceed
original payment amount.')
    WHERE (
        COALESCE((SELECT SUM(refund_amount) FROM refunds WHERE payment_id =
NEW.payment_id), 0.00) + NEW.refund_amount
    ) > (SELECT amount FROM payments WHERE payment_id = NEW.payment_id);
END;

-- Trigger: Enforce Cumulative Refund Ceiling on UPDATE
CREATE TRIGGER trg_check_refund_limit_update
BEFORE UPDATE OF refund_amount, payment_id ON refunds
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Integrity Violation: Cumulative refund amount cannot exceed
original payment amount.')
    WHERE (
        COALESCE((SELECT SUM(refund_amount) FROM refunds WHERE payment_id =
NEW.payment_id AND refund_id != NEW.refund_id), 0.00) + NEW.refund_amount
    ) > (SELECT amount FROM payments WHERE payment_id = NEW.payment_id);
END;

-- =====
-- REPORTING VIEWS
-- =====

-- View 1: Show Occupancy
CREATE VIEW view_show_occupancy AS
SELECT
    s.screening_id,
    c.name AS cinema_name,
    sc.screen_number,
    sc.screen_type,
    m.title AS movie_title,
    s.show_date,
    s.start_time,
    sc.seating_capacity AS total_seats,
    COUNT(bs.booked_seat_id) AS booked_seats,
    (sc.seating_capacity - COUNT(bs.booked_seat_id)) AS available_seats,
    ROUND((COUNT(bs.booked_seat_id) * 100.0 / sc.seating_capacity), 2) AS
occupancy_percentage
FROM screenings s
JOIN screens sc ON s.screen_id = sc.screen_id
JOIN cinemas c ON sc.cinema_id = c.cinema_id
JOIN movies m ON s.movie_id = m.movie_id
LEFT JOIN booked_seats bs ON s.screening_id = bs.screening_id AND bs.status = 'Booked'
GROUP BY s.screening_id, c.name, sc.screen_number, sc.screen_type, m.title, s.show_date,
s.start_time, sc.seating_capacity;

-- View 2: Movie Performance
CREATE VIEW view_movie_performance AS
SELECT
    m.movie_id,
    m.title AS movie_title,
    m.genre,
    m.language,
    COUNT(DISTINCT s.screening_id) AS total_screenings,
    COUNT(t.ticket_id) AS tickets_sold,
    COALESCE(SUM(bs.price), 0.00) AS box_office_gross
FROM movies m

```

```

LEFT JOIN screenings s ON m.movie_id = s.movie_id
LEFT JOIN bookings b ON s.screening_id = b.screening_id AND b.booking_status !=
'Cancelled'
LEFT JOIN booked_seats bs ON b.booking_id = bs.booking_id AND bs.status = 'Booked'
LEFT JOIN tickets t ON bs.booked_seat_id = t.booked_seat_id AND t.status != 'Cancelled'
GROUP BY m.movie_id, m.title, m.genre, m.language;

```

-- View 3: Screen Utilization

```

CREATE VIEW view_screen_utilization AS
SELECT

```

```

    c.name AS cinema_name,
    sc.screen_id,
    sc.screen_number,
    sc.screen_type,
    sc.seating_capacity,
    COUNT(s.screening_id) AS total_shows_scheduled,
    COALESCE(SUM(m.duration_minutes) / 60.0, 0.0) AS operational_hours_runtime,
    COALESCE(SUM(CASE WHEN bs.status = 'Booked' THEN 1 ELSE 0 END), 0) AS
total_admissions,
    ROUND(COALESCE(SUM(CASE WHEN bs.status = 'Booked' THEN 1 ELSE 0 END) * 100.0 /
NULLIF(COUNT(s.screening_id) * sc.seating_capacity, 0), 0), 2) AS
average_capacity_utilization
FROM screens sc
JOIN cinemas c ON sc.cinema_id = c.cinema_id
LEFT JOIN screenings s ON sc.screen_id = s.screen_id AND s.status != 'Cancelled'
LEFT JOIN movies m ON s.movie_id = m.movie_id
LEFT JOIN booked_seats bs ON s.screening_id = bs.screening_id
GROUP BY c.name, sc.screen_id, sc.screen_number, sc.screen_type, sc.seating_capacity;

```

-- View 4: Concession Sales Summary

```

CREATE VIEW view_concession_sales_summary AS
SELECT

```

```

    ci.item_id,
    ci.item_name,
    ci.category,
    ci.unit_price,
    ci.stock_quantity AS current_stock,
    COALESCE(SUM(oi.quantity), 0) AS total_units_sold,
    COALESCE(SUM(oi.subtotal), 0.00) AS total_revenue
FROM concession_items ci
LEFT JOIN order_items oi ON ci.item_id = oi.item_id
LEFT JOIN orders o ON oi.order_id = o.order_id AND o.order_status != 'Cancelled'
GROUP BY ci.item_id, ci.item_name, ci.category, ci.unit_price, ci.stock_quantity;

```

-- View 5: Cancellations and Refunds

```

CREATE VIEW view_cancellations_and_refunds AS
SELECT

```

```

    c.cancellation_id,
    b.booking_id,
    cust.full_name AS customer_name,
    m.title AS movie_title,
    s.show_date,
    b.total_amount AS booking_amount,
    c.cancellation_fee,
    r.refund_amount,
    c.reason,
    c.cancellation_time,
    r.refund_reference
FROM cancellations c
JOIN bookings b ON c.booking_id = b.booking_id
JOIN customers cust ON b.customer_id = cust.customer_id
JOIN screenings s ON b.screening_id = s.screening_id
JOIN movies m ON s.movie_id = m.movie_id

```

```

LEFT JOIN refunds r ON c.cancellation_id = r.cancellation_id;

-- View 6: Total Revenue Breakdown
CREATE VIEW view_total_revenue_breakdown AS
SELECT
    c.name AS cinema_name,
    COALESCE(SUM(CASE WHEN p.booking_id IS NOT NULL AND p.payment_status = 'Success'
THEN p.amount ELSE 0 END), 0.00) AS ticket_revenue,
    COALESCE(SUM(CASE WHEN p.order_id IS NOT NULL AND p.payment_status = 'Success' THEN
p.amount ELSE 0 END), 0.00) AS concession_revenue,
    COALESCE(SUM(canc.cancellation_fee), 0.00) AS cancellation_penalties_retained,
    COALESCE(SUM(ref.refund_amount), 0.00) AS total_refunds_disbursed,
    (
        COALESCE(SUM(CASE WHEN p.booking_id IS NOT NULL AND p.payment_status = 'Success'
THEN p.amount ELSE 0 END), 0.00) +
        COALESCE(SUM(CASE WHEN p.order_id IS NOT NULL AND p.payment_status = 'Success'
THEN p.amount ELSE 0 END), 0.00) +
        COALESCE(SUM(canc.cancellation_fee), 0.00) -
        COALESCE(SUM(ref.refund_amount), 0.00)
    ) AS net_operating_revenue
FROM cinemas c
LEFT JOIN screens sc ON c.cinema_id = sc.cinema_id
LEFT JOIN screenings s ON sc.screen_id = s.screen_id
LEFT JOIN bookings b ON s.screening_id = b.screening_id
LEFT JOIN payments p ON b.booking_id = p.booking_id
LEFT JOIN cancellations canc ON b.booking_id = canc.booking_id
LEFT JOIN refunds ref ON canc.cancellation_id = ref.cancellation_id
GROUP BY c.name;

```

APPENDIX B - APPLICATION SOURCE CODE

The complete, syntactically verified Python application source code is provided below and archived in `project/app/cinema_app.py` (incorporating correct arithmetic, validated variable identifiers, and Pythonic parameter bindings):

```

def book_tickets(customer_id, screening_id, seat_ids, payment_method="UPI"):
    if not seat_ids:
        return False, "Error: No seats selected for booking."

    conn = get_connection()
    cur = conn.cursor()
    try:
        conn.execute("BEGIN TRANSACTION;")

        # 1. Fetch screening ticket price
        cur.execute("SELECT ticket_price FROM screenings WHERE screening_id = ?",
(screening_id,))
        row = cur.fetchone()
        if not row:
            raise ValueError("Screening does not exist.")
        unit_price = row[0]

        # 2. Check if any selected seat is already booked
        cur.execute("""
            SELECT seat_id FROM booked_seats
            WHERE screening_id = ? AND seat_id IN ({} ) AND status = 'Booked'
            """.format(', '.join('? ' * len(seat_ids)), [screening_id] + seat_ids)
        already_booked = cur.fetchall()
        if already_booked:
            conflicted = [str(r[0]) for r in already_booked]

```

```

        raise ValueError(f"Seat(s) {', '.join(conflicted)} already booked! Double
booking prevented.")

    total_amount = unit_price * len(seat_ids)

    # 3. Create booking header
    cur.execute("""
        INSERT INTO bookings (customer_id, screening_id, total_amount,
booking_status)
        VALUES (?, ?, ?, 'Confirmed')
        """, (customer_id, screening_id, total_amount))
    booking_id = cur.lastrowid

    # 4. Insert booked seats & issue tickets with SHA-256 security hash
    issued_tickets = []
    for sid in seat_ids:
        cur.execute("""
            INSERT INTO booked_seats (booking_id, screening_id, seat_id, price,
status)
            VALUES (?, ?, ?, ?, 'Booked')
            """, (booking_id, screening_id, sid, unit_price))
        booked_seat_id = cur.lastrowid

        tkt_number = f"TKT-BKG{booking_id}-S{sid}-
{datetime.datetime.now().strftime('%y%m%d')}"
        barcode = hashlib.sha256(f"{tkt_number}-
{booked_seat_id}".encode()).hexdigest()[:16]
        cur.execute("""
            INSERT INTO tickets (booking_id, booked_seat_id, ticket_number,
barcode_hash, status)
            VALUES (?, ?, ?, ?, 'Active')
            """, (booking_id, booked_seat_id, tkt_number, barcode))
        issued_tickets.append((tkt_number, barcode, sid))

    # 5. Insert payment transaction
    txn_ref = f"TXN-{payment_method[:3].upper()}-{booking_id}-
{datetime.datetime.now().strftime('%H%M%S')}"
    cur.execute("""
        INSERT INTO payments (booking_id, order_id, amount, payment_method,
transaction_reference, payment_status)
        VALUES (?, NULL, ?, ?, ?, 'Success')
        """, (booking_id, total_amount, payment_method, txn_ref))
    payment_id = cur.lastrowid

    conn.commit()
    conn.close()
    return True, {
        "booking_id": booking_id,
        "total_amount": total_amount,
        "payment_id": payment_id,
        "transaction_reference": txn_ref,
        "tickets": issued_tickets
    }
except Exception as e:
    conn.rollback()
    conn.close()
    return False, str(e)

# -----
# MODULE 4: CONCESSION ORDERING (STOCK DEDUCTION & BILLING)
# -----
def list_concessions():
    conn = get_connection()

```

```

cur = conn.cursor()
cur.execute("SELECT item_id, item_name, category, unit_price, stock_quantity FROM
concession_items")
rows = cur.fetchall()
conn.close()
header("Concession Menu & Live Stock")
print(f"{'ID':<4} | {'Item Name':<34} | {'Category':<10} | {'Price':<8} |
{'Stock':<6}")
print("-" * 74)
for r in rows:
    print(f"{r[0]:<4} | {r[1]:<34} | {r[2]:<10} | Rs.{r[3]:<5.0f} | {r[4]:<6}")

```

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

System verification logs, execution outputs, and sample transaction runs have been archived in project/sql/cinema_booking.db.